

The recursive clause below of some predicate is designed with backward recursion because:

The recursive clause below of some predicate is designed with backward recursion because:

```
p([H|T],Some_Result):-  
    p(T, Some_Part_Result),  
    combines(H, Some_Part_Result,Some_Result).
```

Select one:

- ☐ a. It is NOT backward recursion as combines has 3 arguments
- ☒ b.  
It processes the current element after ending the recursion
- ☐ c. No other operation is performed before the recursive call
- ☐ d. It only has 2 arguments, hence, cannot be forward

A cut never affects:

A cut NEVER affects:

Select one:

- ☐ a. Its parent node
- ☐ b. A sibling node to its left
- ☒ c.  
Subtrees of subgoals to its right
- ☐ d. The subtree of a sibling node to its left

Which categories of nodes are frozen(prevented from backtracking) by a cut:

Which categories of nodes are frozen (prevented from backtracking) by a cut:

Select one:

- ☐ a. The parent node if the cut is the last subgoal
- ☐ b.  
Its parent node after the execution of the cut
- ☒ c. Any node to its left before the execution of the cut
- ☐ d. Any sibling node to its right after the execution of the cut

Consider a predicate having a recursive clause as the one below:

Consider a predicate having a recursive clause as the one below:

```
p([H|T], Some_Result, End_Result):-  
    combines(H, Something, New_Some_Result),  
    p(T, New_Some_Result, End_Result).
```

Using this strategy, the result would have to be initialized:

Select one:

- ☐ a. On the stopping condition, in the second argument
- ☐ b. When the initial call is made, in the third argument
- ☐ c. On the stopping condition, in the third argument
- ☒ d.  
When the initial call is made, in the second argument

The following matching:  $f(A, g(B), 4)$  with  $f(a, g, 4)$  would fail because:

The following matching:  $f(A, g(B), 4)$  with  $f(a, g, 4)$  would fail because:

Select one:

- ☐ a. 4 is the 3rd argument
- ☐ b. A should be lower case
- ☒ c. Because of the second argument ( $g(B)$  is an expression while  $g$  is a constant)
- ☐ d. The matching actually succeeds

Which of the following clause heads will the query: `?- process([a,b,c], variable, Constant).` NOT match?

Which of the following clause heads will the query: `?- process([a,b,c], variable, Constant).` NOT match?

Select one:

- ☐ a. `process([A,B,C], C, v):-`
- ☐ b. `process([a_b, C], v, C):-`
- ☒ c. `process([A,B,c_], V, c):-`
- ☐ d. `process([a|T], C, v):-`

Given the compound term  $n(c(A,B), v, m(1, [2,3]))$ , which of the following terms it CANNOT unify with, and why?

Given the compound term  $n(c(A,B), v, m(1, [2,3]))$ , which of the following terms it CANNOT unify with, and why?

Select one:

- ☐ a.  $n(A,B,C)$ , because not all three arguments can be variable
- ☒ b.  $N(c(a,B), v, m(1, [2,3]))$ , because the name of the structure has to be constant
- ☐ c.  $n(c(a,B), v, m(1, [2,B]))$ , because the unifications involving  $B$  are in contradiction
- ☐ d.  $N(c(a,B), v, m(1, [H|T]))$ , because the third arguments do not unify

When matching  $[X|Y]$  with  $[1,2,b]$  the result is:

When matching  $[X|Y]$  with  $[1,2,b]$  the result is:

Select one:

- ☒ a.  
Success, with  $X=1$ ,  $Y=[2,b]$
- ☐ b.  
Failure, as  $Y$  cannot become heterogeneous
- ☐ c. Failure, as the first list has 3 elements while the second 2 elements
- ☐ d. Success, with  $X=[1,2]$ ,  $Y=b$

Consider a predicate having a recursive clause, as below:

Consider a predicate having a recursive clause, as below:

```
p([H|T], Some_Result):-  
    p(T, Some_Part_Result),  
    combines(H, Some_Part_Result, Some_Result).
```

Using this strategy:

Select one:

- ☐ a. The result is built as recursion returns, in `Some_Part_Result`, using the value from `Some_Result`
- ☐ b. The result is built as recursion progresses, in `Some_Result`, using the value from `Some_Part_Result`
- ☒ c.  
The result is built as recursion returns, in `Some_Result`, using the value from `Some_Part_Result`
- ☐ d. The result is built as recursion progresses, in `Some_Part_Result`, using the value from `Some_Result`

The following matching:  $[A,b|C]$  with  $[1,2,3,4]$  fails because:

The following matching: [A,b|C] with [1,2,3,4] fails because:

Select one:

- ☒ a.  
Both b and 2 are constants (and different)
- ☐ b. The first list has 3 elements while the second one 4 elements
- ☐ c.  
The first list has letters while the second one numbers
- ☐ d. The matching succeeds with instantiation: A=1, C=[3,4]

Consider a predicate having a recursive clause as the one below:

Consider a predicate having a recursive clause as the one below:

```
p([H|T], Some_Result, End_Result):-  
    combines(H, Something, New_Some_Result),  
    p(T, New_Some_Result, End_Result).
```

Using this strategy, the result "increases":

Select one:

- ☒ a. As each recursive call returns, in the second argument
- ☐ b. With each new recursive call, in the third argument
- ☐ c. As each recursive call returns, in the third argument
- ☐ d.  
With each new recursive call, in the second argument

Which of the answers from below is correct and the most specific?:

Which of the answers from below is correct and the most specific?

The list `[A, _]` may consist of ...

Select one:

- ☐ a. exactly 1 element
- ☒ b. at least 1 element
- ☐ c. at least 2 elements
- ☒ d. exactly 2 elements

[Clear my choice](#)

The following predicate should search for the first occurrence of an element in an incomplete list:

The following predicate should search for the first occurrence of an element in an *incomplete list*. It:

```
member(X, [X|_]) :- !.  
member(X, _):- var(_),!,fail.  
member(X, [_|T]):-member(X,T).
```

Select one:

- ☐ a. is not correct, because it fails when reaching the variable at the end of the list
- ☐ b. is not correct, because it never finds the searched element
- ☐ c. is correct, because the condition for reaching the variable at the end of the list is explicit, and contains definitive failure
- ☒ d. is not correct, because the clause for reaching the variable at the end of the list is not the first

Which of the answers from below is correct and the most specific?:

Which of the answers from below is correct and the most specific?

The list `[A,B|_]` may consist of ...

Select one:

- ☒ a. at least 2 elements
- ☐ b. exactly 2 elements
- ☐ c. at least 3 elements
- ☐ d. exactly 3 elements

The following predicate is assumed to check if the tree is BST:

The following predicate is assumed to `check if the tree is BST` (Binary Search Tree):

```
is_BST(nil) .  
is_BST(t(t(LL,KL,LR),K,t(RL,KR,RR))):-  
    KL<K<KR,!,  
    is_BST(t(LL,KL,LR)),  
    is_BST(t(RL,KR,RR)).
```

Choose the most specific (and correct) answer from below:

Select one:

- ☐ a. the predicate is incorrect as there is no failure clause
- ☐ b. the predicate is incorrect, as the inequality check should be reversed
- ☐ c. the predicate is correct
- ☒ d. the predicate is incorrect, as the inequality check should consider other keys

Sey we want to write a predicate which adds an element at the end of some incomplete list, given as an argument. The predicate should:



Say we want to write a predicate which adds an element at the end of some *incomplete list*, given as argument. The predicate should:

Select one:

- ☐ a. traverse the given list, instantiate an additional result variable accordingly when reaching the end of the list, then copy all the input list elements in the result, as recursion returns, to maintain their order
- ☐ b. traverse the given list, to reach the variable at the end, then instantiate that variable accordingly
- ☒ c. traverse the given list, and add the elements of the input list, one by one, in an additional result argument, as recursion proceeds, making sure to add the element at the end of the result list (when recursion ends)
- ☐ d. directly instantiate the variable at the end of the given list, accordingly (no traversal needed)

Consider a predicate having a recursive clause as the one below:

Consider a predicate having a recursive clause, as below:

```
p([H|T], Some_Result):-  
    p(T, Some_Part_Result),  
    combines(H, Some_Part_Result, Some_Result).
```

On the stopping condition:

Select one:

- ☐ a. The result has to be made available at the top level (call level)
- ☒ b. The result has to be initialized
- ☐ c. The value of the second argument doesn't matter
- ☐ d. We obtain the final result

I want to write a predicate which partitions the elements of a list into two lists.....

I want to write a predicate which partitions the elements of a list into two lists - the first containing elements smaller than the pivot, and the second containing the elements larger than or equal to the pivot. Complete the predicate specification below according to the requirements:

`p(H, [X|T], [X|Sm], Lg):-X<H, !, p(H, T, Sm, Lg).`

`p(H, [X|T], Sm, [X|Lg]):-p(H, T, Sm, Lg).`

`p(_, [], [], []).`



Complete the predicate below such that it performs the merge operation on two ordered lists:

Complete the predicate below such that it performs the merge operation on two ordered lists:

```

merge([H1|T1], [H2|T2], [H1|R]) :- H1 < H2, !, merge([H2|T2], R).
merge(L, [H2|T2], [H2|R]) :- merge(L, T2, R).
merge([], L, L).
merge(L, [], L).
?-merge([1,4,8], [2,5,8], L).
L=[1,2,4,5,8,8];
no
  
```

Which of the following assert sequence can generate the given clause order for the dynamic predicate `insect/1`:

Which of the following assert sequences can generate the given clause order for the dynamic predicate `insect/1`:

```

insect(ant).
insect(bee).
insect(moth).
insect(fly).
insect(mosquito).
  
```

Select one:

- ☐ a. `assert(insect(ant)) -> assert(insect(fly)) -> assert(insect(mosquito))`
- ☒ b. `asserta(insect(ant)) -> assert(insect(fly)) -> asserta(insect(bee)) -> assert(insect(mosquito))`
- ☐ c. `assert(insect(mosquito)) -> asserta(insect(fly)) -> asserta(insect(moth)) -> asserta(insect(ant))`
- ☐ d. `assertz(insect(ant)) -> assertz(insect(fly)) -> assertz(insect(mosquito))`

Which of the following clause heads will the query: `?-process(a,_b,c).` NOT match?

Question 1

Not yet answered

Marked out of 1.00

Flag question

Which of the following clause heads will the query: `?- process(a,_b,c).` NOT match?

Select one:

- ☐ a. `process(a,D,C):-`
- ☐ b. `process(A,B,C):-`
- ☒ c. `process(a, d, c):-`
- ☐ d. `process(A,B,c):-`

A cut would freeze(prevent from backtracking) some nodes in the tree. However, it is NOT freezing:

PL2021 / 22 March - 28 March / QUIZZ ENG

Question 4

Not yet answered

Marked out of 1.00

Flag question

A cut would freeze (prevent from backtracking) some nodes in the tree. However, it is NOT freezing:

Select one:

- ☒ a. A sibling node to its left before the cut is executed
- ☐ b. The subtree of its leftmost sibling node after the cut is executed
- ☐ c. Its parent node after the cut is executed
- ☐ d. Its leftmost sibling node after the cut is executed

Next page

Quiz1 seria A

Jump to...

5. Sorting

Say we want to write a predicate which receives as input a list of integers, and produces as output a list containing their squares, in reverse order. To obtain such...

Say we want to write a predicate which receives as input a list of integers, and produces as output a list containing their squares, in **reverse order**. To obtain such a predicate (correct and complete), the recursive clause should look like:

Select one or more:

- ☐ a. `square([H|T], R):- square(T,R).`
- ☐ b. `square([H|T], [NH|R]):- NH is H*H, square(T,R).`
- ☐ c. `square([H|T], R):- NH is H*H, square(T,[HN|R]).`
- ☒ d. `square([H|T], R):- NH is H*H, square(T,[HN|R],R).`

The predicate below is assumed to replace exactly one occurrence (in turn) of the first argument, from the list given in the second argument, with the third argument, to generate the list in the fourth argument.

The predicate below is assumed to replace **exactly one** occurrence (in turn) of the first argument, from the list given in the second argument, with the third argument, to generate the list in the fourth argument.

To behave accordingly (as in the associated example), the code should be updated in the following way:

```
rep(X, [X|T], Y, [Y|T]).
rep(X, [H|T], Y, [H|R]):-
    rep(X, T, Y, R).
rep(_, [], _, []).
?- rep(b, [a,b,c,b,d], z, R).
R= [a,z,c,b,d]?
R= [a,b,c,z,d]?
no
```

Select one:

- ☐ a. Update the clause 2 as `rep(X,[H|T],Y,R):-!,rep(X,T,Y,R).`
- ☐ b. Update the clause 1 as `rep(X,[X|T],Y,[Y|T]):-!.`
- ☐ c. Swap the order of clauses 1 and 2.
- ☐ d. Update the clause 2 as `rep(X,[H|T],Y,[H|R]):-!,rep(X,T,Y,R).`
- ☐ e. Swap the order of clauses 1 and 3.
- ☒ f. Remove the clause `rep([],[],[]).`
- ☐ g. Remove the clause `rep(X,[X|T],Y,[Y|T]).`
- ☐ h. Add another clause like: `rep(X,[X],Y,[Y]).`



In search operation in an incomplete structure the following condition must hold true:

Question 2

Not yet answered

Marked out of 1.00

Flag question

In a search operation in an incomplete structure the following condition must hold true:

Select one:

- ☐ a. have explicit success condition(s) in the beginning immediately followed by explicit failure condition(s)
- ☐ b. have explicit failure condition(s) or success condition(s)
- ☒ c. have explicit failure condition(s) in the beginning immediately followed by success condition(s)
- ☐ d. have explicit failure condition(s) and success condition(s)

Fill in the gaps below such as to obtain a predicate which swaps adjacent numbers in a list, if their order is incorrect:

Fill in the gaps below such as to obtain a predicate which swaps adjacent numbers in a list, if their order is incorrect (the first larger than the second):

```

one_pass([H1, H2|T], [H2|R]) :- H1 > [H1|T], one_pass([H1|T], R).
one_pass([H1|T], [H1|R]) :- one_pass(T, R).
one_pass([], []).

```

?- one\_pass([3,1,4,5,2], R).  
R=[1,3,4,2,5];  
no.

Which of the answers from below is correct and the most specific?:

Question 5

Not yet answered

Marked out of 1.00

Flag question

Which of the answers from below is correct and the most specific?

The list [A, B, \_] may consist of ...

Select one:

- ☐ a. exactly 2 elements
- ☐ b. at least 2 elements
- ☒ c. exactly 3 elements
- ☐ d. at least 3 elements

[Clear my choice](#)

If we want a decreasing sorting by selection strategy(below), where max and delete are correct predicates to find the max value, and delete.....

Question 1

Not yet answered

Marked out of 1.00

Flag question

If we want a **decreasing** sorting by selection strategy (below), where `max` and `delete` are correct predicates to find the max value, and `delete` a value from a list respectively, which condition must hold true (choose the most specific one):

```
sort(In, [M|Out]) :-  
    max(In, M),  
    delete(M, In, Int),  
    sort(Int, Out).  
  
sort([], []).
```

Select one:

- ☒ a. `delete` needs to delete the first occurrence of the value from the list
- ☐ b. `min` needs to be implemented with forward recursion
- ☐ c. `max` needs to choose the first occurrence of the largest value from the list
- ☐ d. `min` needs to use a strict inequality

[Clear my choice](#)

The following sequence is assumed to be used in a condition-controlled loop bubble sort. Would it be a sound choice.....

The following sequence is assumed to be used in a condition-controlled loop (while or repeat-until) bubble sort. Would it be a sound choice (that is, can bubble sort be implemented using it)?

```
traverse_once([A,B|T], [B|R]) :-  
    A >= B, !,  
    traverse_once([A|T], R).  
  
traverse_once([A|T], R) :-  
    traverse_once(T, R).  
  
traverse_once([], []).
```

Select one:

- ☐ a. No! It (`traverse_once`) fails as fact should come as first clause.
- ☒ b. Yes! If properly employed, that is, until no swap occurred in the latest `traverse_once`.
- ☐ c. No! It (`traverse_once`) fails as it cannot reach the clause with the empty list due to patterns in the other clauses.
- ☒ d. No! A bubble sort using it might enter an infinite loop in case duplicates exist in the input.

Which of the answers from below is correct and the most specific?:

Which of the answers from below is correct and the most specific?

The list `[A|_]` may consist of ...

Select one:

- ☐ a. exactly 2 elements
- ☐ b. exactly 1 element
- ☒ c. at least 1 element
- ☐ d. at least 2 elements

The following sequence is assumed to be used in a condition-controlled loop bubble sort for an input without duplicates....

The following sequence is assumed to be used in a condition-controlled loop (while or repeat-until) bubble sort for an input **without duplicates**. Would it be a sound choice (that is, can be bubble sort implemented using it)?

```
traverse_once([A,B|T], [A|R]) :-  
    A < B, !,  
    traverse_once([B|T], R),  
    traverse_once([A,B|T], [B|R]) :-  
    traverse_once([A|T], R),  
    traverse_once([A], [A]).
```

Select one:

- ☒ a. Yes! If properly employed, that is, until no swap occurred in the latest `traverse_once`.
- ☐ b. Yes, but only if case `traverse_once` is called `n` times (where `n` is the length of the input).
- ☐ c. No! A bubble sort using it might enter an infinite loop.
- ☐ d. No! It (`traverse_once`) fails as it never reaches the empty list.

If the code below uses `append` with the `fact` as the clause, for the input `[3,5,7,4,6]` what would be the second selection of `A` and `B` for swapping during execution?

If the code below uses `append` with the `fact` as the first clause, for the input `[3,5,7,4,6]` what would be the second selection of `A` and `B` for swapping (setting `B,A`) during execution?

```
bubbleSort(In, Out) :-  
    append(X, [A, B|Y], In),  
    A > B, !,  
    append(X, [B, A|Y], Int),  
    bubbleSort(Int, Out),  
    bubbleSort(In, In).
```

Select one:

- ☐ a. A=5, B=7
- ☐ b. A=7, B=6
- ☐ c. A=4, B=7
- ☒ d. A=5, B=4



The predicate below should decreasingly sort by selection, using min and delete correct predicates to find the min value, and delete a value from a list.....

The predicate below should **decreasingly** sort by selection, using min and delete correct predicates to find the min value, and delete a value from a list, respectively. Fill in the blanks for a correct predicate:

```

wrapper(In, Out) :-
    sort(In, [ ], Out).

sort(In, Acc, [ ], Out) :-
    min(In, M, In, Int),
    delete([ ], M, [ ], [ ]),
    NA = [M|Acc],
    sort(Int, [ ], Out),
    sort([ ], NA, Out).

```

The following predicate should search for the first occurrence of an element in a incomplete list:

The following predicate should search for the first occurrence of an element in an *incomplete list*. It:

```

member(X, L) :- var(L), fail.
member(X, [_|_]) :- !.
member(X, [_|T]) :- member(X, T).

```

Select one:

- ☐ a. is not correct, because it is deterministic
- ☐ b. is not correct, because it never finds the searched element
- ☐ c. is correct, because the condition for reaching the end of the list is the first, and contains definitive failure
- ☒ d. is not correct, because backtracking is allowed when reaching the variable at the end of the list

Say we want to write a predicate which inserts an element, in order, into an ordered list, as last occurrence...

2

d  
out of  
question

Say we want to write a predicate which inserts an element, in order, into an ordered list, as **last occurrence**. Complete the

you get a correct predicate:

insertOrd(E, [], [E]).

insertOrd(E, [H|T], [E,H|T]) :- E < H, !.

insertOrd(E, [H|T], [H|R]) :- insertOrd(E, T, R).

Quiz 4 EN

Jump to...

10.0

The following sequence is assumed to insert a key in an incomplete BTS. Choose the most specific answer from below:

The following sequence is assumed to insert a key in an incomplete BST. Choose the most specific answer from below:

```
ins(t(L, Key1, _), Key) :-
    Key1 > Key, !,
    ins(L, Key).
ins(t(_, Key1, R), Key) :-
    ins(R, Key).
ins(t(_, Key, _), Key) :- !.
```

Select one:

- ☐ a. The predicate is incorrect, as it's missing the output argument
- ☐ b. The predicate is incorrect as it does not treat the case when the incomplete tree (var (T)) is reached.
- ☐ c. The predicate is incorrect as the second clause is missing the comparison Key1 < Key.
- ☒ d. The predicate is incorrect as the last clause must be the first one.

In a decreasing sort by selection strategy(below ), the max\_delete predicate is a stable predicate which correctly deletes the maximal value from the list. If we print Out, what would be the second printed list on input In =.....

In a decreasing sorting by selection strategy (below), the max\_delete predicate is a stable predicate which correctly deletes the maximal value from the list. If we print Out, what would be the second printed list on input In = [3,2,4,1,5]?

```
sort(In, [M|Out]) :-
    max_delete(M, In, Int),
    sort(Int, Out),
    sort([], []).
```

Select one:

- ☐ a. [1,2]
- ☐ b. [5]
- ☒ c. [1]
- ☐ d. [4,5]

[Clear my choice](#)

The predicate below should be a predicate to delete the last occurrence of the max value from a list. Fill in the blanks for a correct predicate:

The predicate below should be a predicate to delete the **last** occurrence of the max value from a list. Fill in the blanks for a correct predicate:

```
max_delete([H|T], , ) :-
    max_delete(T, Max, DT),
    H ,
    ,
    max_delete([H|T], , ) .
```

The following sequence is assumed to insert a key in a incomplete BTS. Choose the most specific answer from below:

The following sequence is assumed to insert a key in an incomplete BST. Choose the most specific answer from below:

```
ins(t(_,Key,_),Key):-!.
ins(t(L,Key1,R),Key):-
    var(R),
    Key1>Key,!,
    ins(L,Key).
ins(t(L,Key1,R),Key):-
    var(L),
    !,
    ins(R,Key).
```

Select one:

- ☐ a. The predicate is incorrect as it does not treat the case when the incomplete tree (`var(T)`) is reached.
- ☐ b. The predicate is incorrect as the third clause is missing the comparison `Key1>Key` and the first clause must be the last one.
- ☐ c. The predicate is incorrect, as it's missing the output argument
- ☒ d. The predicate is incorrect as it does not handle the general case (with both `L` and `R` `nonvar`).

The following sequence is assumed to search for a key in an incomplete BST. Choose the most specific answer from below:

The following sequence is assumed to search for a key in an incomplete BST. Choose the most specific answer from below:

```
search(T,_):-var(T),!,fail.
search(t(_,Key,_),Key):-!.
search(t(L,Key1,R),Key):-
    search(L,Key), search(R,Key).
```

Select one:

- ☐ a. The predicate is incorrect as the first clause should be with `!, var(T), fail.` in the body.
- ☒ b. The predicate is incorrect. The last clause should contain a disjunction.
- ☐ c. The predicate is incorrect as it is never comparing keys.
- ☐ d. The predicate is incorrect: clauses 1 and 2 should be in opposite order.

The following sequence is assumed to insert a key in an incomplete BST. . Choose the most specific answer from below:

Question 5

Not yet answered

Marked out of 1.00

Flag question

The following sequence is assumed to insert a key in an incomplete BST. Choose the most specific answer from below:

```
ins (t (_, Key, _), Key) :- !.
```

```
ins (t (L, Key1, _), Key) :-
```

```
    Key1 > Key, !,
```

```
    ins (L, Key).
```

```
ins (t (_, Key1, R), Key) :-
```

```
    ins (R, Key).
```

Select one:

☐ a. The predicate is correct.

☒ b.

The predicate is incorrect as it does not treat the case when the incomplete tree ( $\text{var } t$ ) is reached).

☐ c. The predicate is incorrect, as it's missing the output argument

☐ d.

The predicate is incorrect as the second clause must be the first one.

You are given the predicate below and the example graph on the right (undirected, edge-clause form, nodes appear in lexicographic order):

You are given the predicate below and the example graph on the right (undirected, *edge-clause* form, nodes appear in lexicographic order):

```
d(X):- assertz(vert(X)), edge(X,Y), \+(vert(Y)), d(Y).
```

What will the predicate base look like after the execution of:

?- d(a).

Select one or more:

☐ a. vert(d).

vert(c).

vert(b).

vert(a).

☐ b. vert(e).

vert(f).

vert(d).

vert(c).

vert(b).

vert(a).

☒ c. vert(a).

vert(b).

vert(c).

vert(d).

vert(f).

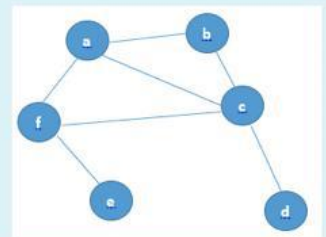
vert(e).

☐ d. vert(a).

vert(b).

vert(c).

vert(d).



The following code is assumed to collect, in order, the internal nodes with 2 children of a incomplete binary search tree...

The following code is assumed to collect, in order, the internal nodes with 2 children of an incomplete binary search tree. Choose the most specific answer from below:

```
collect_2_children(t(L,_,R),N):-var(L),!,
    collect_2_children(R,N).
collect_2_children(t(L,_,R),N):-var(R),!,
    collect_2_children(L,N).
collect_2_children(t(L,Key,R),NN):-
    collect_2_children(R,NR),
    collect_2_children(L,NL),
    append(NL,[Key|NR],NN).
collect_2_children(t(L,_,R),[]):-
    var(L),var(R),!.
```

Select one:

- ☒ a. The predicate is incorrect as the last clause should be the first one.
- ☐ b. The predicate is correct.
- ☐ c. The predicate is incorrect as the clause right subtree is processed before the left one.
- ☐ d. The predicate is incorrect, as is missing a clause on `var(T)`.

In an undirected graph represented by its edges, the predicat checking from below is assumed to check.....

In an undirected graph represented by its edges (e.g. `is_edge(a,b)`), the predicate checking from below is assumed to check that each vertex in the graph is possessing a certain property which is soundly checked by the `verify(Vertex)` predicate. Assuming that `verify` is correctly implemented, choose the most specific answer from below:

```
checking:- is_edge(X,_),
    not(v(X)),
    asserts(v(X)),
    not(verify(X)),
    fail.
checking.
```

Select one:

- ☒ a. The predicate is correct if `!` is added after `not(verify(X))`.
- ☐ b. The predicate is correct as it is.
- ☐ c. The predicate is correct if `asserts` is replaced by `asserta`.
- ☐ d. The predicate is correct if the negation (`not`) in front of (`verify`) is removed.

In an undirected graph represented by its edges, the predicate constructing from below is assumed to check.....

In an undirected graph represented by its edges (e.g. `is_edge(a,b)`), the predicate `constructing` from below is assumed to build an additional knowledge base of good nodes (`good(Node)`) containing all nodes of the graph for which a certain property holds true. Assuming that `verify(Vertex)` predicate is correctly implemented, and it succeeds if and only if `Vertex` is possessing the underlying property, choose the most specific answer from below:

```
constructing:- is_edge(X,_),
               not(v(X)),
               assertz(v(X)),
               verify(X),
               assertz(good(X)),
               fail.
constructing.
```

Select one:

- ☐ a. The predicate is correct if the predicate `verify` is negated (`not(verify)`).
- ☒ b. The predicate is correct as it is.
- ☐ c. The predicate is correct if `assertz` is replaced by `asserta`.
- ☐ d. The predicate is correct if the `!` is added after `verify`.

The following code is assumed to collect, in order, the internal nodes(non leaves) of an incomplete BST, using difference lists.....

The following code is assumed to collect, in order, the internal nodes (non leaves) of an incomplete BST, using difference lists (argument 2 is the beginning and argument 3 is the end of the difference list). Choose the most specific answer from below:

```
collect_internal(t(L_,R),B,B):-
    var(L),var(R),!.
collect_internal(t(L,Key,R),B,[Key|E]):-
    var(R),!,
    collect_internal(L,B,E).
collect_internal(t(L,Key,R),[Key|B],E):-
    var(L),!,
    collect_internal(R,B,E).
collect_internal(t(L,Key,R),BRes,ERes):-
    collect_internal(R,Int,ERes),
    collect_internal(L,BRes,[Key|Int]).
```

Select one:

- ☐ a. The predicate is incorrect, as is missing a first clause on `var(T)`.
- ☐ b. The predicate is incorrect because the pattern of adding the `Key` in the second clause (on the left list `(L)`) `[Key|E]` must occur in the recursive call.
- ☒ c. The predicate is incorrect because the pattern of adding the `Key` in the third clause (on the right list `(R)`) `[Key|B]` must occur in the recursive call.
- ☐ d. The predicate is incorrect as in the 4<sup>th</sup> clause the recursive calls are swapped.



Complete the predicate below such that it finds a path in a graph, having the length smaller than M:

Complete the predicate below such that it finds a path in a graph, having the length smaller than M:

```

p(X,Y,M,P):- p(X,Y,M,[X],P).
p(X,  ,M, PP,PP):-
  length( , L),
   < M.
p(X,  ,M, PP, FP):-
  edge(X,Z),
  \+(member( , PP)),
  p( ,Y, M,  , FP).
  
```

Complete the predicate below such that it finds a path in a directed graph of length greater than M:

1 / 24 May - 30 May / Quiz6 seria B

Question 3  
Not yet answered  
Marked out of 1.00  
Flag question

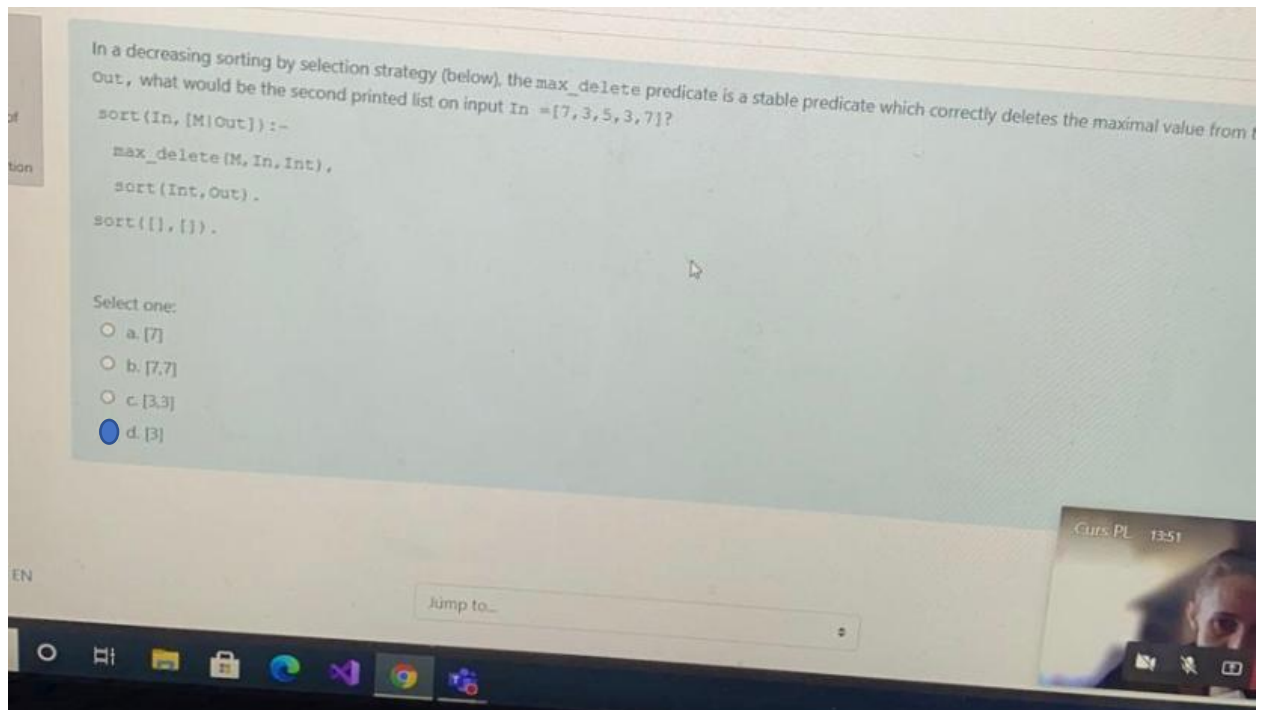
Complete the predicate below such that it finds a path in a directed graph of length greater than M:

```

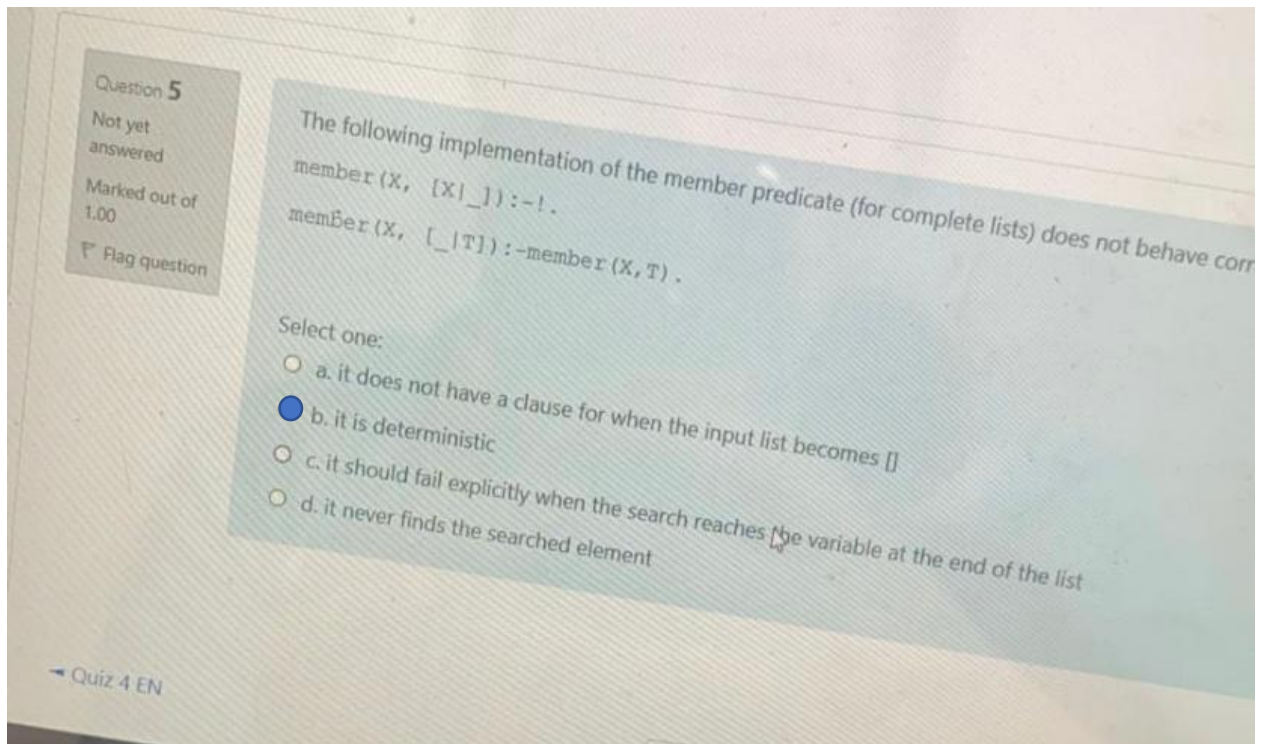
p(X,Y,M,P):- p(X,Y,M,[X],P).
p(X,X,M, PP,PP):-
  length( , L),
   > M.
p(X,Y,M, PP, FP):-
  edge(X,Z),
  \+(member( , PP)),
  p( ,Y, M,  , FP).
  
```

Jump to...

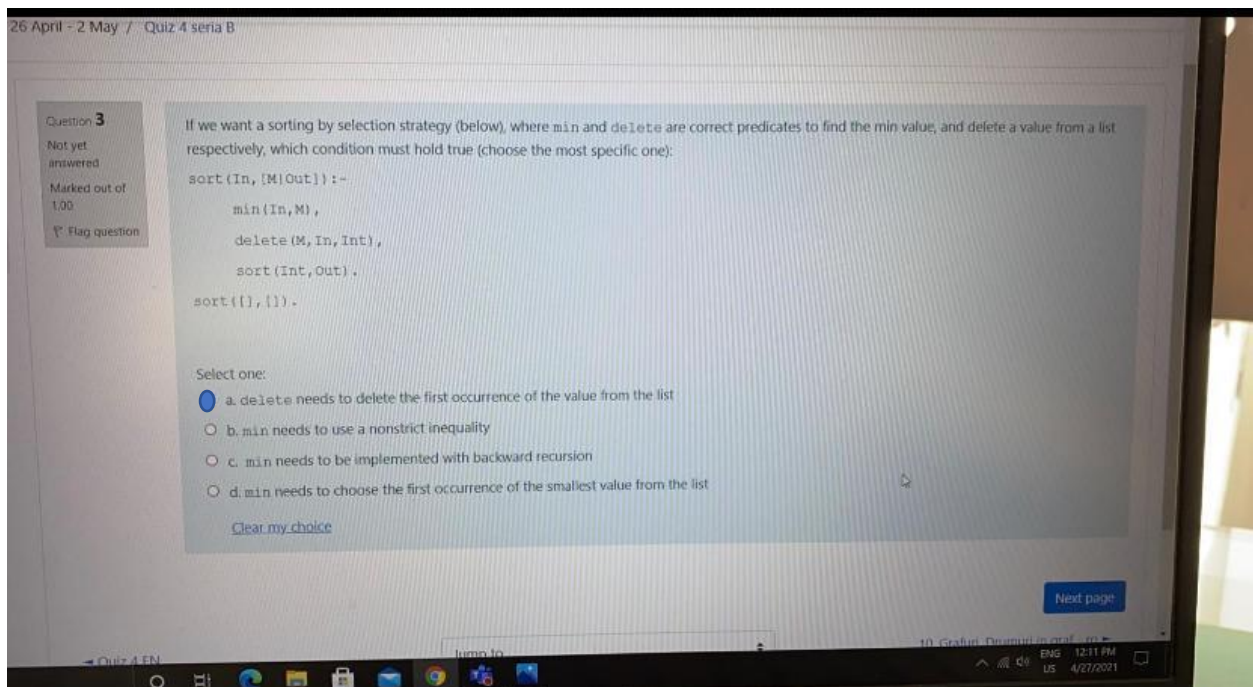
In a decreasing sort by selection strategy (below), the `max_delete` predicate is a stable predicate which correctly deletes the maximal value from the list. If we print Out, what would be the second printed list on input In =.....

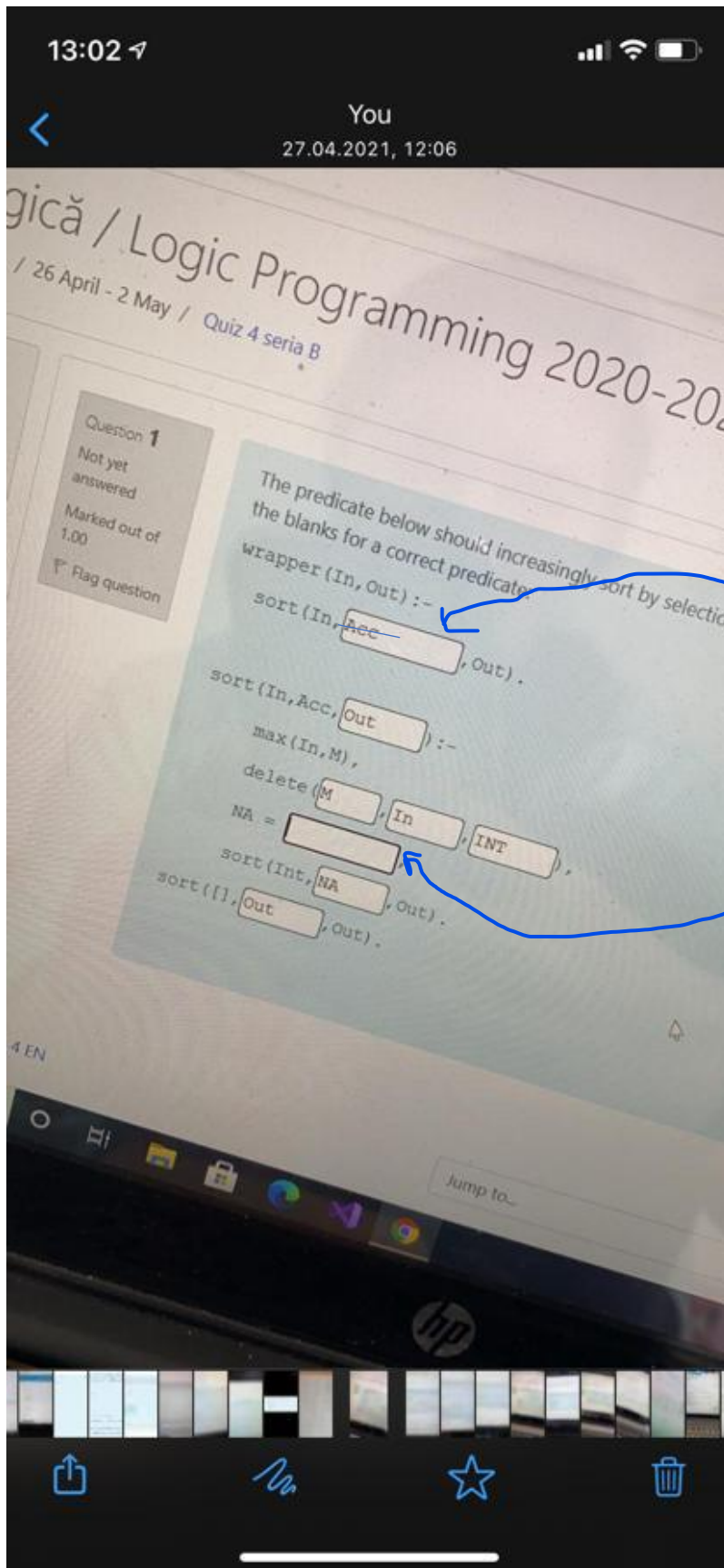


The following implementation of the member predicate (for complete lists) does not behave.....



If we want a decreasing sorting by selection strategy (below), where min and delete are correct predicates to find the min value, and delete.....





The predicate below should increasingly sort by selection....

# Logic Programming 2020-2021

/ 26 April - 2 May / Quiz 4 seria B

Question 1  
Not yet answered  
Marked out of 1.00  
Flag question

The predicate below should increasingly sort by selection. Fill in the blanks for a correct predicate.

sort(In, Acc, Out) :-  
wrapper(In, Out).

sort(In, Acc, Out) :-  
max(In, M),

delete(M, In, INT),  
NA = ,  
sort(INT, NA, Out).

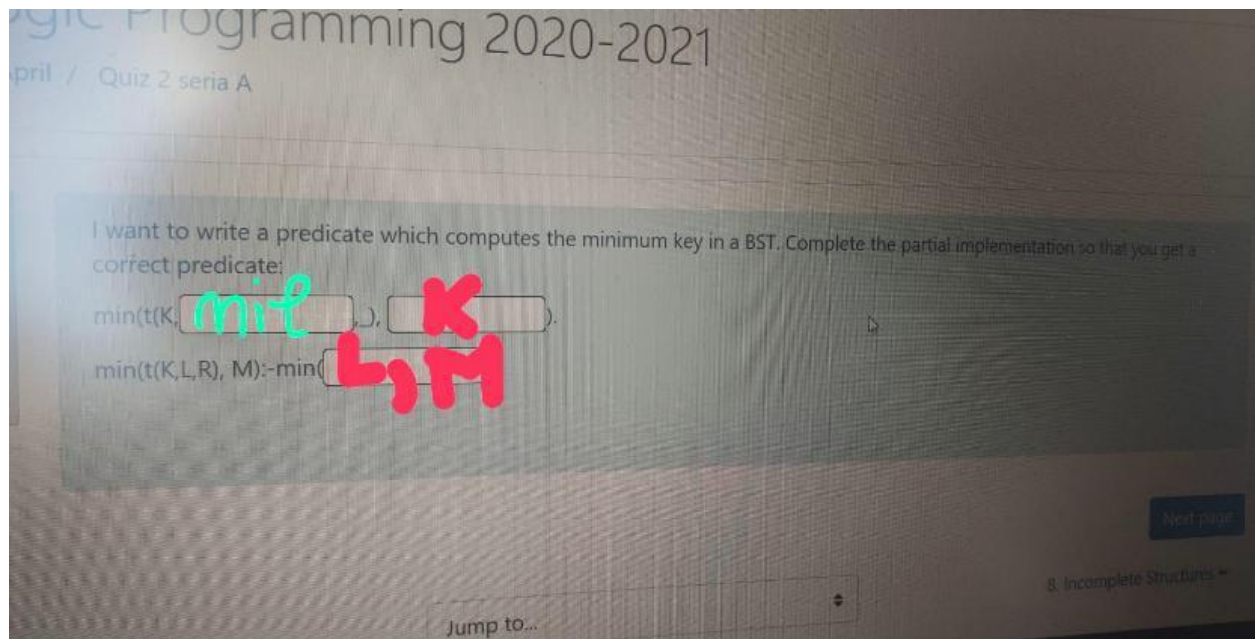
sort([], Out, Out).

[]

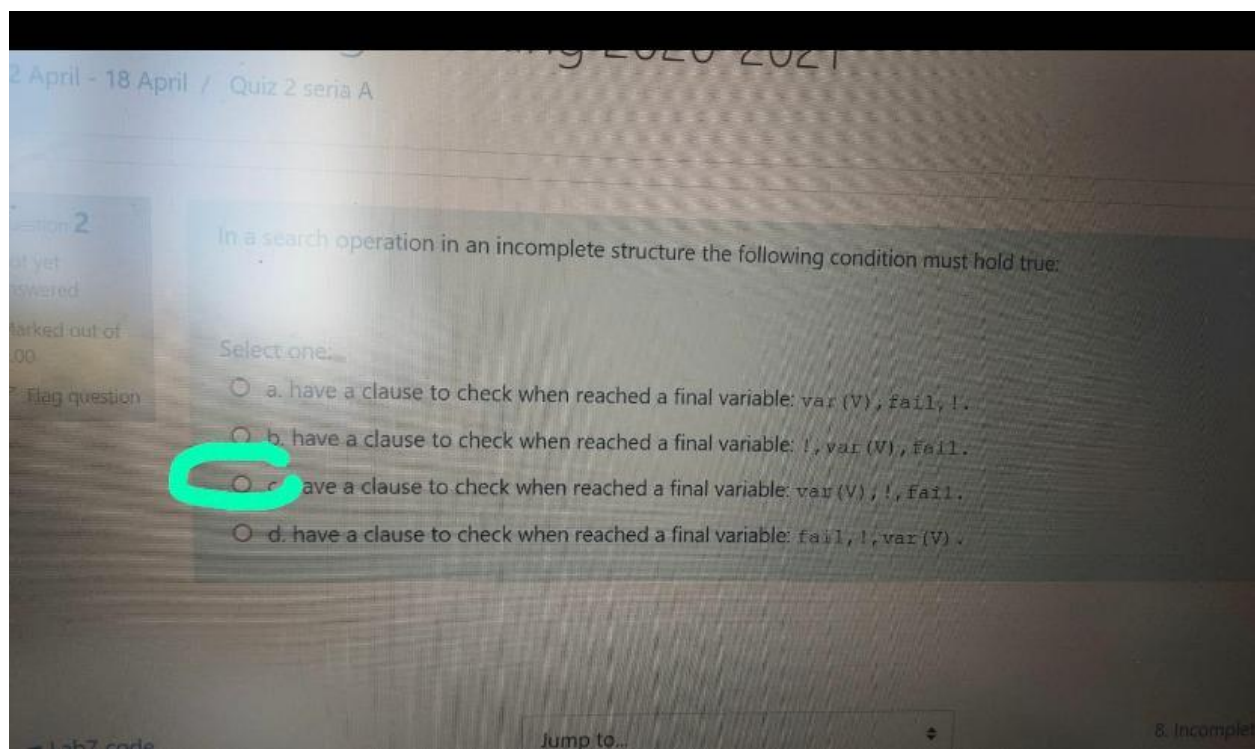
[M|Acc]



I want to write a predicate which computes the minimum key in a BST. Complete the partial implementation so that you get a correct predicate:



In search operation in an incomplete structure the following condition must hold true:



Say we want to write a predicate which receives as input a list of integers, and produces as output a list containing their squared roots, in reverse order....

Question 4  
Not yet answered  
Marked out of 1.00  
Flag question

Say we want to write a predicate which receives as input a list of integers, and produces as output a list containing their squared roots, in **reverse order**. To obtain such a predicate (correct and complete), the recursive clause should look like:

Select one or more:

- ☐ a. `square([H|T], [NH|R]) :- NH is sqrt(H), square(T,R).`
- ☒ b. `square([H|T], R) :- NH is sqrt(H), square(T,[NH|R],R).`
- ☐ c. `square([H|T], R) :- NH is sqrt(H), square(T,[NH|R]).`
- ☐ d. `square([H|T], R) :- square(T,R).`

Next page

Quiz2 ENG Jump to... 5. Sorting

The following matching:  $[1,2,3]$  with  $[A,B|C]$  would provide the following instantiations:

28 March / Quiz2 seria B

Question 3  
Not yet answered  
Marked out of 1.00  
Flag question

The following matching:  $[1,2,3]$  with  $[A,B|C]$  would provide the following instantiations:

Select one:

- ☒ a.  $A=1, B=2, C=[3]$
- ☐ b. No instantiation is provided
- ☐ c.  $A=1, B=2, C=3$
- ☐ d. The matching fails

[Clear my choice](#)

Quiz2 ENG Jump to...

as Cristiana Tintean (Log out)

ASUS VivoBook

The following predicate deletes all occurrences of an element from a list:

Question 3  
Complete  
Mark 1.00 out of 1.00  
Flag question

The following predicate deletes all occurrences of an element from a list:

$d(X, [X|T], R) :- d(X, T, R).$

$d(X, [H|T], [H|R]) :- d(X, T, R).$

$d(\_, [], []).$

Modify it such that it deletes only the first occurrence.

Select one or more:

- ☐ a. Clause 2 should become:  $d(X, [H|T], R) :- d(X, T, R).$
- ☒ b. Clause 1 should become:  $d(X, [X|T], T) :- !.$
- ☐ c. Clause 1 should become:  $d(X, [X|T], T).$
- ☐ d. Clause 1 should become:  $d(X, [X|T], R) :- !, d(X, T, R).$

The correct answer is: Clause 1 should become:  $d(X, [X|T], T) :- !.$

Given the predicates below:

Question 4  
Complete  
Mark 1.00 out of 1.00  
Flag question

Given the predicates below:

$a([], L, L).$

$a([H|T], L, [H|R]) :- a(T, L, R).$

$aa(A, B, C, D) :- a(B, C, E), a(A, E, D).$

How many instantiations of C (not necessarily distinct) do we get by repeating the query:

?- aa(, C, [1,2,3]).

Select one or more:

- ☐ a. 4
- ☐ b. No answer. Execution enters infinite loop.
- ☐ c. 3
- ☒ d. 10

The correct answer is: 10

Complete the predicate below such that it deletes all the occurrences of an element from a list:

I want to write a predicate which generates the list of even integers from a given list. Complete...

Question 1  
Complete  
Mark 1.00 out of 1.00  
Flag question

Complete the predicate below such that it deletes all the occurrences of an element from a list :

$delete\_all(X, [X \text{ } ], T, R) :- delete\_all(X, T, R \text{ } ).$

$delete\_all(X, [H \text{ } ], [H \text{ } R]) :- delete\_all(X, T, R \text{ } ).$

$delete\_all(\_, [ \text{ } ], [ \text{ } ]).$

Question 2  
Complete  
Mark 1.00 out of 1.00  
Flag question

I want to write a predicate which generates the list of even integers from a given list. Complete the predicate specification below accordingly:

$even([H|T], R \text{ } ) :- 1 \text{ is } H \bmod 2, ! \text{ } , even(T, R).$

$even([H|T], [H|R] \text{ } ) :- even(T \text{ } , R).$

$even([ \text{ } ], [ \text{ } ]).$

?- even([4,1,7,2,3], R).

R=[4,2];

no.



Given the following predicate specification:

Which is the answer to the following query:

Question 1

Complete

Mark 1.00 out of 1.00

Flag question

Given the following predicate specification:

$p(\text{nil}, []).$

$p(t(K,L,R),\text{List}) :- p(L,LL), p(R,LR), \text{append}(LL, [K|LR], \text{List}).$

Which is the answer to the following query:

?-  $p(t(12,t(7,t(3,\text{nil},\text{nil}),\text{nil}),t(16,t(14,\text{nil},\text{nil}),t(19,t(17,\text{nil},\text{nil}),\text{nil}))), L).$

Select one or more:

- ☐ a. yes,  $L = [3, 7, 14, 16, 17, 19, 12];$   
no.
- ☐ b. yes,  $L = [3, 7, 14, 16, 17, 19];$   
no.
- ☒ c. yes,  $L = [3, 7, 12, 14, 16, 17, 19];$   
no.
- ☐ d. yes,  $L = [19, 17, 16, 14, 12, 7, 3];$   
no.

The correct answer is: yes,  $L = [3, 7, 12, 14, 16, 17, 19];$   
no.

Complete the following predicate such that it performs the transformation of an incomplete list to a complete list.

I have the deterministic member predicate for shallow complete lists, as:

Question 2

Complete

Mark 1.00 out of 1.00

Flag question

Complete the following predicate such that it performs the transformation of an incomplete list into a complete list:

$t(L) :- \text{var}(L), !, L = [].$

$t([_T]) :- t(T).$

Question 3

Complete

Mark 1.00 out of 1.00

Flag question

I have the deterministic member predicate for shallow complete lists, as:

$m(X, [X\_]) :- !.$

$m(X, [_T]) :- m(X, T).$

Modify the predicate such that it finds all the occurrences of a given element in a deep list (i.e. non-deterministic, member in deep lists).

Select one or more:

- ☐ a. Add, as second clause:  $m(X, [H\_]) :- m(X, H).$
- ☐ b. Change the second clause to:  $m(X, [H|T]) :- m(X, H), m(X, T).$  and remove  $!$  from the first clause.
- ☒ c. Add clause:  $m(X, [H\_]) :- m(X, H).$  and remove  $!$  from the first clause.
- ☐ d. Change the second clause to:  $m(X, [H|T]) :- m(X, H), !, m(X, T).$

The correct answer is: Add clause:  $m(X, [H\_]) :- m(X, H).$  and remove  $!$  from the first clause.

To make the predicate `select_sort/2` stable,....

To make the predicate `select_sort/2` stable, we need to

```
select_sort(In, [Min|Rest]):-  
    min_delete(In, In_without_Min, Min),  
    select_sort(In_without_Min, Rest).  
select_sort([], []).
```

```
min_delete([H|T], [H|R], M):-
```

```
    min_delete(T, R, M),
```

```
    M<=H, !.
```

```
min_delete([H|T], T, H).
```

Select one:



a.

change the inequality into a strict one



b. nothing, it is stable as it is



c. swap the order of `min_delete(In, In_without_Min, M)`



d. add a clause for the empty list in `min_delete` predicate

[Clear my choice](#)

To make the predicate `ins_sort/3` stable, we need to:

Quiz navigation

1 2 3 4 5

Finish attempt ...

Time left 0:03:04

Question 4  
Not yet answered  
Marked out of 1.00  
Flag question

To make the predicate `ins_sort/3` stable, we need to:

```
ins([A|T], B, [A|R]) :-  
    B >= A, !,  
    ins(T, B, R).  
ins(L, B, [B|L]).
```

```
ins_sort([A|T], Int, Out) :-  
    ins(Int, A, NewInt),  
    ins_sort(T, NewInt, Out).  
ins_sort([], Out, Out).
```

Select one:

- ☐ a. swap the order of `ins(Int, A, NewInt)` and `ins_sort(T, NewInt, Out)`
- ☐ b. nothing, it is stable as it is
- ☒ c. change the inequality into a strict one
- ☐ d. add a clause for the empty list in `ins/3` predicate

Se considera predicatul :

Cate solutii are urmatoarea intrebare:

Question 4  
Answer saved  
Marked out of 1.00  
Flag question

Se considera predicatul:

`s([H|T], L, [H|R]) :- s(T, L, R).`

`s([], R, R).`

Cate solutii (i.e. raspunsuri 'yes') are urmatoarea intrebare:

?- `s([], [B|_], [1,2,3,4])`.

|

Select one or more:

- ☒ a. 5
- ☐ b. 0
- ☒ c. 4
- ☐ d. 1

Urmatorul predicat realizeaza reuniunea a doua multimi(reprezentate ca si liste; predicatul `member` se considera cunoscut si implementat):

Question 3  
Answer saved  
Marked out of 1.00  
Flag question

Urmatorul predicat realizeaza reuniunea a doua multimi (reprezentate ca si liste; predicatul `member` se considera cunoscut si implementat):

`u([H|T], L, R) :- member(H, L), !, u(T, L, R).`

`u([H|T], L, [H|R]) :- u(T, L, R).`

`u([], L, L).`

Alegeti dintre urmatoarele optiuni, pe cea care modifica corespunzator predicatul, asa incat sa realizeze INTERSECTIA a doua multimi, in loc de reuniunea lor.

Select one or more:

- ☐ a. Capul clauzei 1 devine: `u([H|T], L, [H|R]) :- ...`
- ☒ b. Clauza 3 devine: `u([], _, []).`
- ☒ c. Capul clauzei 1 devine: `u([H|T], L, [H|R]) :- ...`, capul clauzei 2 devine: `u([H|T], L, R)` si clauza 3 devine: `u([], _, []).`
- ☒ d. Corpul clauzei 1 devine: `\+ member(H, L), !, u(T, L, R).`

Completati predicatul de mai jos astfel incat sa obtineti inlocuirea unei aparitii a elementului Old cu elementu New, in lista data ca argument:

Question 2

Answer saved

Marked out of 1.00

Flag question

Completati predicatul de mai jos astfel incat sa obtineti inlocuirea unei aparitii a elementului Old cu elementul New, in lista data ca argument:

replace(Old, New, [Old|T], [New|T]).

replace(Old, New, [H|T], [H|R]) :- replace(Old, New, T, R).

replace(, , , ).

Completati predicatul de mai jos pentru a obtine cautarea unui element intr- o lista:

Question 1

Answer saved

Marked out of 1.00

Flag question

Completati predicatul de mai jos pentru a obtine cautarea unui element intr-o lista:

member(X, [X|\_]).

member(X, [\_|T]) :- member(X, T).

The predicate below is intending to search for a given element in an incomplete Binary Search Tree....

Question 2

Incorrect

Mark 0.00 out of 1.00

Flag question

The predicate below is intending to search for a given element in an incomplete Binary Search Tree (ended in variable). Do do so, we need to do the following:

search\_iBST(K, t(, K, )) :- !.

search\_iBST(SK, t(L, K, )) :-

SK < K, !,

search\_iBST(SK, L).

search\_iBST(SK, t(, , R)) :-

search\_iBST(SK, R).

Select one:

☐ a.

add the following clause search\_iBST(, V) :- var(V), !. mandatory as the first clause.

☒ b.

add the following clause search\_iBST(, V) :- var(V), !, fail. mandatory as the first clause.

☐ c.

add the following clause search\_iBST(, V) :- var(V), !, fail. anywhere in the code (first, second, third or fourth clause) ✖

☐ d.

add some clause for nil.

Your answer is incorrect.

Which of the answers from below is correct and the most specific.

Which of the answers from below is correct and the most specific?

The list [ ] is ...

Select one:

☐ a.

an incomplete list.

☐ b.

not an incomplete list as it is missing the | pattern.

☐ c.

not an incomplete list as there are no other elements in the list other than the final variable.

☒ d.

not an incomplete list as it contains only one single variable element.

## ANA – INTREBAREA 31 DIN ARHIVA Siminei

Completati predicatul de mai jos astfel incat sa traverseze arborele in **post-ordine** folosind liste diferenta.

**Question 1**  
Partially correct  
Mark 0.80 out of 1.00  
Flag question

Completați predicatul de mai jos astfel încât să traverseze arborele în **post-ordine** folosind liste diferență.

`p_dl(nil, R`  `).`

`p_dl(t(K,L,R)`  `,LE):-p_dl(L, LS,`  `, p_dl(R`  `,ML,`

`LE`  `).`

Predicatul de mai jos trebuie sa inverseze o lista folosind liste diferenta.

Ce trebuie modificat?

**Question 2**  
Correct  
Mark 1.00 out of 1.00  
Flag question

Predicatul de mai jos trebuie să inverseze o listă folosind liste diferență.

`reverse([],_).`  
`reverse([H|T],Rs,Re):-reverse(T,Rs,[H|Re]).`

Ce trebuie modificat?

Select one or more:

- ☐ a. Modifică clauza 2 în: `reverse([H|T],[H|Rs],Re):-reverse(T,Rs,Re).` și clauza 1 în: `reverse([],L,L).`
- ☐ b. Modifică clauza 2 în: `reverse([H|T],Rs,[H|Re]):-reverse(T,Rs,Re).` și clauza 1 în: `reverse([],L,L).`
- ☐ c. Schimbă ordinea clauzelor.
- ☒ d. Modifică clauza 1 în: `reverse([],L,L).` ✓

Se da urmatoarea secventa de intrebari:

?-assertz(n(1)), assertz(n(2)), asserta(n(3)).

?-retract(n(X)),!,fail.

Cum va arata baza de cunostinte dupa executia celor 2 intrebari?

**Question 3**  
Incorrect  
Mark 0.00 out of 1.00  
Flag question

Se da următoare secvență de întrebări:

?- assertz(n(1)), assertz(n(2)), asserta(n(3)).

?- retract(n(X)),!, fail.

Cum va arăta baza de cunoștințe după execuția celor 2 întrebări?

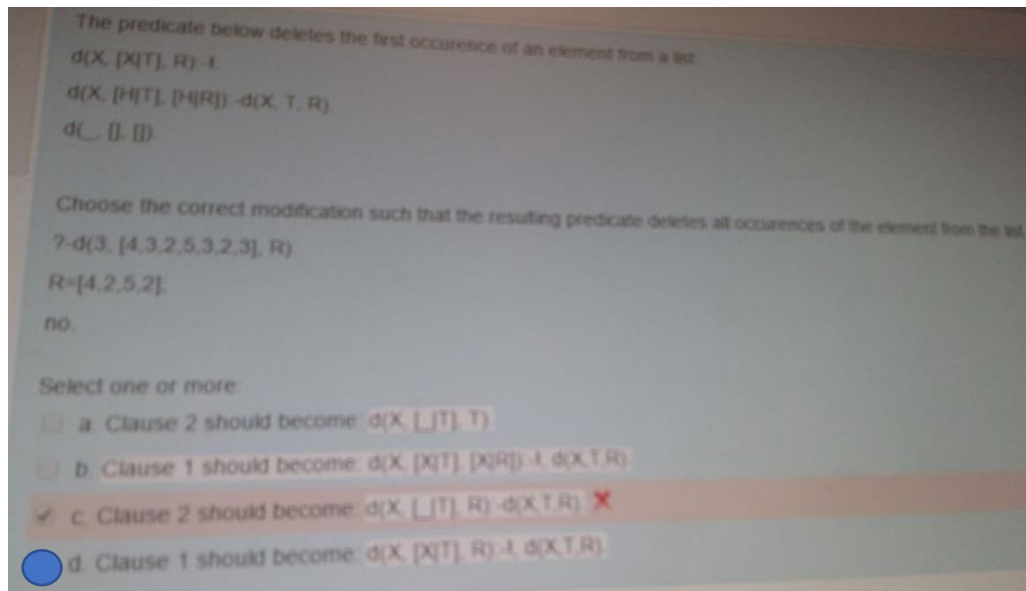
Select one or more:

- ☒ a. Nimic ✗
- ☐ b. n(2).  
n(3).
- ☐ c. n(2).  
n(1).
- ☐ d) n(1). n(2).



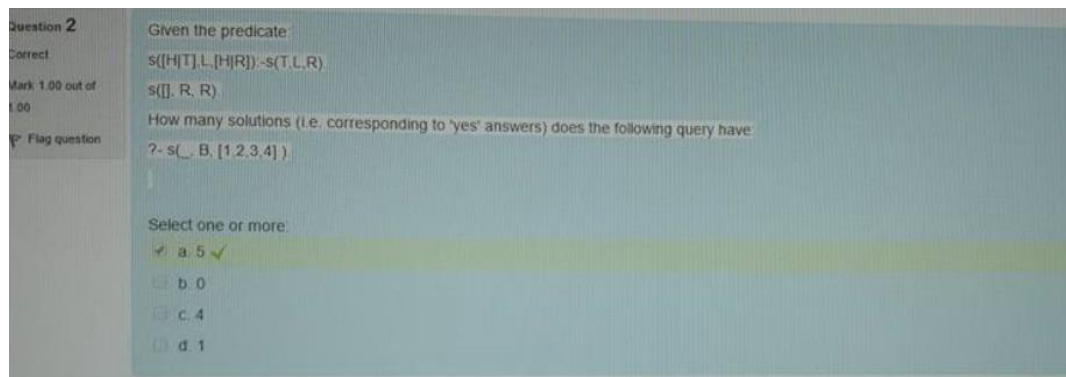
The predicate below deletes the first occurrence of an element from a list.

Choose the correct modification such that the resulting predicate deletes all occurrence of the element from the list.



Given the predicate: [...]

How many solutions (i.e. corresponding to 'yes' answers) does the following query have:



The following predicate performs the union of two sets (represented as lists, member is considered known and implemented): [...]

Select from the alternatives below the one which correctly modifies the predicate such that it performs the intersection of two sets, instead of the union.



**Question 3**  
Correct  
Mark 1.00 out of 1.00  
Flag question

The following predicate performs the union of two sets (represented as lists, member is considered known and implemented):

```
u([H|T],L,R) :- member(H,L),!,u(T,L,R).
u([H|T],L,[H|R]) :- u(T,L,R).
u([],L,L).
```

Select from the alternatives below the one which correctly modifies the predicate such that it performs the intersection of two sets, instead of the union.

Select one or more:

- ☐ a. Change clause 1 head to: `u([H|T],L,[H|R]) :-`
- ☐ b. Change clause 3 to: `u([],L,L)`
- ☒ c. Change clause 1 head to: `u([H|T],L,[H|R]) :-`, the head of clause 2 to `u([H|T],L,R)` and change clause 3 to: `u([],L,L)` ✓
- ☐ d. Change clause 1 body to: `! member(H,L), !, u(T,L,R)`

Activate Windows  
Go to Settings

The following predicate should remove duplicates from a list, is it correct? If not, choose the necessary corrections:

**Question 1**  
Correct  
Mark 1.00 out of 1.00  
Flag question

The following predicate should remove duplicates from a list, is it correct? If not, choose the necessary corrections:

```
rem([H|T],R):-member(H,T),!,rem(T,R).
rem([H|T],R):-rem(T,R).
rem([],[]).
```

Try: `rem([1,2,3,2,4,1],R).`  
`R=[3,2,4,1]`  
No

Select one or more:

- ☐ a. Not correct, change clause 1 to: `rem([H|T],R):-member(H,T),!,rem(T,R)`
- ☒ b. Not correct, change clause 2 head to: `rem([H|T],R):-!` ✓
- ☐ c. It is correct
- ☐ d. Not correct, change clause 1 head to: `rem([H|T],R):-`

Given the predicate: [...]

How many solutions (i.e. corresponding to 'yes' answers) does the following query have:

**Question 2**  
Incorrect  
Mark 0.00 out of 1.00  
Flag question

Given the predicate

```
s([H|T],L,[H|R]) :- s(T,L,R).
s([],R,R).
```

How many solutions (i.e. corresponding to 'yes' answers) does the following query have:

```
?- s([],B,[1,2,3,4]).
```

Select one or more:

- ☐ a. 5
- ☒ b. 4 ✗
- ☐ c. 0
- ☐ d. 1

I want to write a predicate which replaces the first (and only the first) occurrence of element Old with element New, in a list. Help me complete the predicate below to achieve this task:

**Question 2**  
Correct  
Mark 1.00 out of 1.00  
Flag question

I want to write a predicate which replaces the first (and only the first) occurrence of element *Old* with element *New*, in a list. Help me complete the predicate below to achieve this task.

```

replace(Old, New, [Old|T], [New|T]).
replace(Old, New, [H|T], [H|R]) :- replace_all(Old, New, T, R).
replace([], []).

```

Given the predicate below:

What is the answer to the query:

**Question 3**  
Correct  
Mark 1.00 out of 1.00  
Flag question

Given the predicate below:

```

f([H|T], Acc, R) :- f(T, [H|Acc], R).
f([], R, R).

```

What is the answer to the query:

```

?- f([1,2,3],[4,5],R).

```

Select one or more:

- ☐ a. yes, R=[4,5,1,2,3]
- ☐ b. yes, R=[1,2,3,4,5]
- ☒ c. yes, R=[3,2,1,4,5]
- ☐ d. yes, R=[4,5,3,2,1]
- ☐ e. no

The following predicate should insert an element in an ordered list. Fill in the gaps to complete its specification (see the example query before you submit your answer !!)

**Question 1**  
Partially correct  
Mark 0.67 out of 1.00  
Flag question

The following predicate should insert an element in an ordered list. Fill in the gaps to complete its specification (see the example query before you submit your answer !!)

```

insert_ord(X, [H|T], R, X) :- X >= H, insert_ord(X, T, R).
insert_ord(X, T, [X|T]).

```

```

?- insert_ord(3, [1,2,3,6], R).
R=[1,2,3,3,6].
no.

```

The following predicate performs the intersection of two sets (represented as lists, member is considered known and implemented):

Select from the alternatives below the one which correctly modifies the predicate such that it performs the difference of two sets, instead of the intersection.

**Question 2**

Incorrect

Mark 0.00 out of 1.00

Flag question

The following predicate performs the intersection of two sets (represented as lists, member is considered known and implemented):

```
i([H|T], L, [H|R]) :- member(H, L), !, i(T, L, R).
```

```
i([H|T], L, R) :- i(T, L, R).
```

```
i([], []).
```

Select from the alternatives below the one which correctly modifies the predicate such that it performs the difference of two sets, instead of the intersection.

Select one or more:

- ☐ a. Change clause 3 to: `i([], L, L).`
- ☒ b. Change clause 1 body to: `!+ member(H, L), !, i(T, L, R).`
- ☒ c. Change clause 1 head to: `i([H|T], L, R) :- ...` ✗
- ☐ d. Change clause 1 head to: `i([H|T], L, R) :- ...` and clause 3 to: `i([], L, L).`

I want to write a predicate which replaces all the occurrences of element *Old* with element *New*, in a list. Help me complete the predicate below to achieve this task.

**Question 3**

Correct

Mark 1.00 out of 1.00

Flag question

I want to write a predicate which replaces all the occurrences of element *Old* with element *New*, in a list. Help me complete the predicate below to achieve this task:

```
replace_all(Old, New, [Old|T], [New|R]) :- replace_all(Old, New, T, R).
```

```
replace_all(Old, New, [H|T], [H|R]) :- replace_all(Old, New, T, R).
```

```
replace_all(_, _, [], []).
```

I want to write a predicate which searches for an element in an incomplete list. Choose the correct option to complete the code below:

**Question 2**

Correct

Mark 1.00 out of 1.00

Flag question

I want to write a predicate which searches for an element in an incomplete list. Choose the correct option to complete the code below.

```
search_ll(X, [_|_]) :- search_ll(X, T).
```

Select one or more:

- ☐ a. Add as first two clauses:  
`search_ll(X, L) :- var(L), !`
- ☐ b. Add as first clause: `search_ll(X, L) :- var(L), !, fail.`
- ☐ c. Add as first clause: `search_ll(X, [_|_]) :- !`
- ☒ d.

Add as first two clauses:

```
search_ll(X, L) :- var(L), !, fail.
```

```
search_ll(X, [_|_]).
```

The following predicate searches for a certain key in an incomplete BST: [...]

How do we have to alter the predicate such that it performs insertion, instead of search, in an incomplete BST?

**Question 3**

Correct

Mark 1.00 out of 1.00

Flag question

The following predicate searches for a certain key in an incomplete BST:

```
search_bt(_, T):-var(T),!,fail.
search_bt(Key,t(Key,_,_)):-!.
search_bt(Key,t(K,L,R)):-Key < K,!,search_bt(Key,L).
search_bt(Key,t(_,_R)):-search_bt(Key,R).
```

How do we have to alter the predicate such that it performs insertion, instead of search, in an incomplete BST?

Select one or more:

- ☐ a. Change the first clause to: `search_bt(_, T):-var(T),!,`
- ☐ b. Remove the first clause and change the second clause to: `search_bt(Key,T):-var(T),T=t(Key,_,_).`
- ☒ c. Remove the first clause. ✓
- ☐ d. Change the first clause to: `search_bt(Key,T):-var(T),!,T=t(Key,_,_),fail`

Activate Windows

Complete the following predicate to compute the height of an incomplete binary tree:

**Question 2**

Correct

Mark 1.00 out of 1.00

Flag question

Complete the following predicate to compute the height of an incomplete binary tree:

```
height(T,0):-var(T),!.
height(t(L,R),H):-height(L,HL),height(R,HR),max(HL,HR),
H is HH+1.
```

```
?- height(t(4,t(2,_),R), R).
R = 2 ;
false
```

The following predicate searches for a certain key in a complete BST:

How do we have to alter the predicate such that it performs insertion, instead of search, in an incomplete BST?

**Question 3**

Correct

Mark 1.00 out of 1.00

Flag question

The following predicate searches for a certain key in a complete BST:

```
search_bt(Key,t(Key,_,_)):-!.
search_bt(Key,t(K,L,R)):-Key < K,!,search_bt(Key,L).
search_bt(Key,t(_,_R)):-search_bt(Key,R).
```

How do we have to alter the predicate such that it performs insertion, instead of search, in an incomplete BST?

Select one or more:

- ☐ a. Change the first clause to: `search_bt(Key,T):-var(Key),!,T=t(Key,_,_).`
- ☒ b. Do nothing. ✓
- ☐ c. Change the first clause to: `search_bt(_, T):-var(T),!,`
- ☐ d. Add, as first clause: `search_bt(Key,T):-var(T),!,T=t(Key,_,_),fail`

I have the predicate below: [...]

How many yes answers do I get?

**Question 1**  
Correct  
Mark 1.00 out of 1.00  
Flag question

I have the predicate below:  
 $s\_L(L) \text{ :- var}(L), !, \text{fail}.$   
 $s(X, [X\_]) \text{ :- !}$   
 $s(X, [\_T]) \text{ :- s}(X, T).$   
 and the query:  
 $? \text{ :- X=3, s}(X, [3,3,3\_]).$   
 How many yes answers do I get?

Select one or more:

- ☒ a. 1 ✓
- ☐ b. 4
- ☐ c. infinite number of yes answers
- ☐ d. 0
- ☐ e. 3

I want to write a predicate which searches for an element in an incomplete list. Choose the correct option to complete the code below:

**Question 2**  
Correct  
Mark 1.00 out of 1.00  
Flag question

I want to write a predicate which searches for an element in an incomplete list. Choose the correct option to complete the code below:

$\text{search\_il}(X, [\_T]) \text{ :- search\_il}(X, T).$

Select one or more:

- ☐ a. Add as first two clauses:  
 $\text{search\_il}(X, L) \text{ :- var}(L), !.$
- ☐ b. Add as first clause:  $\text{search\_il}(X, L) \text{ :- var}(L), !, \text{fail}.$
- ☐ c. Add as first clause:  $\text{search\_il}(X, [X\_]) \text{ :- !}.$
- ☒ d. Add as first two clauses:  
 $\text{search\_il}(X, L) \text{ :- var}(L), !, \text{fail}.$   
 $\text{search\_il}(X, [X\_]).$

Activate W  
Go to Settings

Completati predicatul astfel incat sa insereze un element la sfarsitul listei incomplete.

**Question 1**  
Correct  
Mark 1.00 out of 1.00  
Flag question

Completati predicatul astfel incat sa insereze un element la sfarsitul listei incomplete

$\text{insert\_il}(X, [X\_]) \text{ :- !}$  ✓

$\text{insert\_il}(X, [\_T]) \text{ :- insert\_il}(X, T).$  ✓

Urmatorul predicat verifica daca un element exista intr-o lista completa.

Modificati predicatul astfel incat sa verifice daca un element exista intr-o lista incompleta.

**Question 2**  
Correct  
Mark 1.00 out of 1.00  
Flag question

Urmatorul predicat verifica dacă un element există într-o listă completă:  
 $m(X, [X\_]) \text{ :- !}.$   
 $m(X, [\_T]) \text{ :- m}(X, T).$

Modificati predicatul astfel incat sa verifice daca un element exista intr-o lista incompleta.

Select one or more:

- ☒ a. Adaugă ca primă clauză:  $m(L) \text{ :- var}(L), !, \text{fail}.$  ✓
- ☐ b. Adaugă ca primă clauză:  $m(L) \text{ :- var}(L), !.$
- ☐ c. Adaugă ca primă clauză:  $m(L) \text{ :- var}(L), \text{fail}.$
- ☐ d. Adaugă ca primă clauză:  $m(L) \text{ :- var}(L), !, \text{fail}.$  și șterge "!" din clauza doi.

Avem urmatorul predicat.

Cate raspunsuri de yes/true primesc la intrebarea:

**Question 3**  
Correct  
Mark 1.00 out of 1.00  
Flag question

Avem următorul predicat:

```
s(_ , L):-var(L), fail.  
s(X, [X|_]).  
s(X, [_|T]):-s(X, T).
```

Câte răspunsuri de yes/true primesc la întrebarea:  
?- X=3, s(X,[3,3,3|\_])

Select one or more:

- ☐ a. 4
- ☐ b. 1
- ☐ c. 0
- ☒ d. Infinit ✓
- ☐ e. 3

Completati predicatul astfel incat sa aplatizeze o lista adanca incompleta.

**Question 1**  
Partially correct  
Mark 0.89 out of 1.00  
Flag question

Completați predicatul astfel încât să aplatizeze o listă adâncă incompletă.

```
flatten_ll(L, ) :- var(L) ✓, !.  
flatten_ll([H|T], ) :- atomic(H), , flatten_ll(, R).  
flatten_ll([H|T], ) :- flatten_ll(H, ) ✓, flatten_ll(T, ) ✓, append(R1, R2, ) ✓.
```

Ex apel:  
?- flatten\_ll([1, [2|\_] | \_], R).  
R = [1,2].  
no

Urmatorul predicat verifica daca un element exista intr-o lista completa:

Modificati predicatul astfel incat sa verifice daca un element exista intr-o lista adanca si sa fie nedeterminist.

**Question 2**  
Correct  
Mark 1.00 out of 1.00  
Flag question

Următorul predicat verifică dacă un element există într-o listă completă:

```
m(X, [X|_]) :- !.  
m(X, [_|T]) :- m(X, T).
```

Modificați predicatul astfel încât să verifice dacă un element există într-o listă adâncă și să fie nedeterminist.

Select one or more:

- ☒ a. Adaugă clauza:  $m(X, [H|_]) :- m(X, H)$ , și șterge "!" din prima clauză ✓
- ☐ b. Schimba a doua clauză în:  $m(X, [H|T]) :- m(X, H), !, m(X, T)$ .
- ☐ c. Schimba a doua clauză în:  $m(X, [H|T]) :- m(X, H), m(X, T)$ , și șterge "!" din prima clauză
- ☐ d. Adaugă între prima și a doua clauză:  $m(X, [H|_]) :- m(X, H)$ .

Complete the following predicate such that it computes the depth of a deep list:

**Question 2**  
Partially correct  
Mark 0.83 out of 1.00  
Flag question

Complete the following predicate such that it computes the depth of a deep list:

```
d([], 1).  
d([H|T], D) :- atomic(H), , d(, ) ✓.  
d([H|T], D) :- d(H, D1), d(T, D2), D1 is , max(, , D) ✓.
```



The following predicate has to insert an element at the end of an incomplete list (if it is already in the list, it doesn't get added). Choose the right variant:

**Question 3**  
Incorrect  
Mark 0.00 out of 1.00  
Flag question

The following predicate has to insert an element at the end of an incomplete list (if it is already in the list, it doesn't get added). Choose the right variant:  
`insert_il(X, [_|_]);- insert_il(X, T).`

Select one or more:

- ☐ a. Add, as first clause: `insert_il(X, L):-var(L), !, fail.`
- ☒ b. Add, as first two clauses:  
`insert_il(X, L):-var(L), L=[X|_].`  
`insert_il(X, [X|_]).` ✗
- ☐ c. Add, as first clause: `insert_il(X, [X|_]).`
- ☐ d. Add, as first clause: `insert_il(X, [X|_]):-`

alegeti intrebarea care adauga elementul a la sfarsitul unei liste diferenta ce continue elementele 1,2,3

`q(X,SL,[X|EL],SL,EL).`

`q(a,[1,2,3|L],L,RS,[]).`

Alegeti intrebarea care adauga elementul "a" la sfarsitul unei liste diferenta ce continue elementele 1, 2 si 3.

`q(X, SL,[X|EL], SL,EL).`

Select one or more:

- ☐ a. `?-q(a, [1,2,3], [], RS,RE).`
- ☐ b. `?-q(a, [1,2,3|_], _, RS,RE).`
- ☐ c. `?-q(a, [1,2,3|_], [], RE).`
- ☒ d. `?-q(a, [1,2,3|L], L, RS,[]).` ✓

Se da predicatul de mai jos : `q(X,SL,[X|EL],SL,EL)` . Care este raspunsul la urmatoarea intrebare

`q(3,[1,2,3|L],L,SL,[]).` `L=[3], SL=[1,2,3,3]`

Se da predicatul de mai jos:

`q(X, SL,[X|EL], SL,EL).`

Care este raspunsul la urmatoarea intrebare:

`?-q(3,[1,2,3|L],L,SL,[]).`

Select one or more:

- ☐ a. false
- ☐ b. `SL = [1,2,3,3]`
- ☒ c. `L = [3], SL = [1,2,3,3]` ✓
- ☐ d. `L = [3], SL = [1,2,3]`

Se da urmatoarea secventa de intrebari. Cum va arata baza de cunostinte dupa executia celor 2 intrebari?

`assertz(n(1)),assertz(n(2)),asserta(n(3))`

`retract(n(X)),!,fail.`

Se dă următoare secvență de întrebări:

?- assertz(n(1)), assertz(n(2)), asserta(n(3)).

?- retract(n(X)),!, fail.

Cum va arăta baza de cunoștințe după execuția celor 2 întrebări?

Select one or more:

☒ a. Nimic ✗

☐ b. n(2).  
n(3).

☐ c. n(2).  
n(1).

☐ d. n(1).  
n(2).

Predicatul de mai jos trebuie sa inverseze o lista folosind liste diferenta.

`reverse([],_,_).`

`reverse([H|T],Rs,Re):-reverse(T,Rs,[H|Re]).`

Predicatul de mai jos trebuie să inverseze o listă folosind liste diferență.

`reverse([],_,_).`

`reverse([H|T],Rs,Re):-reverse(T,Rs,[H|Re]).`

Ce trebuie modificat?

Select one or more:

☐ a. Modifică clauza 2 în: `reverse([H|T],[H|Rs],Re):-reverse(T,Rs,Re).`  
și clauza 1 în: `reverse([],L,L).`

☐ b. Modifică clauza 2 în: `reverse([H|T],Rs,[H|Re]):-reverse(T,Rs,Re).`  
și clauza 1 în: `reverse([],L,L).`

☐ c. Schimbă ordinea clauzelor.

☒ d. Modifică clauza 1 în: `reverse([],L,L).` ✓

Completați predicatul de mai jos astfel încât să traverseze arborele în post ordine folosind liste diferență. stg dr key

Completați predicatul de mai jos astfel încât să traverseze arborele în **post-ordine** folosind liste diferență.

```
p_dl(nil, R,  ✓).
p_dl(t(K,L,R),  ✓, LE):-p_dl(L, LS,  ✓), p_dl( ✓, ML,
 ✗).
```

```
inorder_dl(nil,L,L).
inorder_dl(t(K,L,R),LS,LE):-inorder_dl(L,LSL,LEL),inorder_dl(R,LSR,LER),
                             LSL=LSL, LEL=[K|LSR], LE=LER.
```

%post ordine cu liste diferenta

p\_dl(nil,R,R).% lista vida e reprezentata de 2 variabile egale

p\_dl(t(K,L,R),LS,LE):- p\_dl(L,LS,ML),p\_dl(R,ML,[K|LE]).

?- consult(ex1).

true.

?- tree1(T),p\_dl(T,L,[]).

T = t(6, t(4, t(2, nil, nil), t(5, nil, nil)), t(9, t(7, nil, nil), nil)),

L = [2, 5, 4, 7, 9, 6].

Completați următorul predicat astfel încât să se găsească ciclurile dintr-un graf care încep cu nodul X.

Completați următorul predicat astfel încât să găsească ciclurile dintr-un graf care încep cu nodul X.

```
c(X,  ✓, [P]):- arc(X,Y), c(Y,X,[Y],P).
c(X,Y,PP, FP):-
  arc( ✗, Z),
  not(member( ✓, PP)),
  c( ✓, [Z|PP], FP).
c(X,  ✗, PP, P):- reverse(PP, P).
```

Predicatul de mai jos trebuie să găsească primul drum simplu (și doar primul) între 2 noduri din graf (reprezentat cu predicate edge).

```
path(X,Y,Path):-path(X,Y,[X],Path).
```

```
path(X,X,P,P).
```

```
path(X,Y,Path,Res):-edge(X,Z), not(member(Z,Path)),!,path(Z,Y,[Z|Path],Res).
```

```

path(X,Y,Path):-path(X,Y,[X],Path).
path(X,X,P,P).
path(X,Y,Path,Res):-edge(X,Z), not(member(Z,Path)),path(Z,Y,[Z|Path],Res),!.

```

Predicatul de mai jos trebuie să găsească primul drum simplu (și doar primul) între 2 noduri din graf (reprezentat cu predicate edge).

```

path(X,X,P,P).
path(X,Y,Path,Res):-edge(X,Z), not(member(Z,Path)), path(Z,Y,[Z|Path],Res).

```

Ce trebuie modificat?

Select one or more:

- ☐ a. Modifică clauza 2 în: path(X,Y,Path,Res):-edge(X,Z), not(member(Z,Path)),!, path(Z,Y,[Z|Path],Res).
- ☒ b. Modifică clauza 1 în: path(X,X,P,P):-!, și clauza 2 în: path(X,Y,Path,Res):-edge(X,Z), not(member(Z,Path)),!, path(Z,Y,[Z|Path],Res). ❌
- ☐ c. Modifică clauza 1 în: path(X,X,P,P):-!.
- ☐ d. Modifică clauza 2 în: path(X,Y,Path,Res):-edge(X,Z), not(member(Z,Path)), path(Z,Y,[Z|Path],Res), !.

The following predicate should remove duplicates from a list. Is it correct? If not , choose the necessary corrections.

The following predicate should remove duplicates from a list. Is it correct? If not, choose the necessary corrections:

```
rem([H|T], R):-member(H,T),!, rem(T, R).
```

```
rem([H|T], R):-rem(T,R).
```

```
rem([],[]).
```

```
?-rem([1,2,3,2,4,1], R).
```

```
R=[3,2,4,1];
```

```
no.
```

Select one or more:

- ☐ a. Not correct, change clause 1 to: rem([H|T], [H|R]):-member(H,T), rem(T, R).
- ☒ b. Not correct, change clause 2 head to: rem([H|T], [H|R]):- ✓
- ☐ c. It is correct.
- ☐ d. Not correct, change clause 1 head to: rem([H|T], [H|R]):-

Given the predicate

```
s([H|T],L,[H|R]):-s(T,L,R).
```

```
s([],R,R).
```

How many solutions s(\_B,[1,2,3,4]).

```
?- st(_B,[1,2,3,4]).
```

```
B = [] ;
```

```
B = [4] ;
```

```
B = [3, 4] ;
```

```
B = [2, 3, 4] ;
```

B = [1, 2, 3, 4].

Given the predicate:

`s([H|T], L, [H|R]) :- s(T, L, R).`

`s([], R, R).`

How many solutions (i.e. corresponding to 'yes' answers) does the following query have:

`?- s(_, B, [1,2,3,4]).`

Select one or more:

☐ a. 5

☒ b. 4 ✗

☐ c. 0

☐ d. 1

Fill in the gaps below such as to obtain a predicate which swaps adjacent numbers in a list, if their order is incorrect (the first larger than the second). Use H whenever you need a new variable for an element from the list:

Fill in the gaps below such as to obtain a predicate which swaps adjacent numbers in a list, if their order is incorrect (the first larger than the second). Use H whenever you need a NEW variable for an element from the list:

`one_pass([H1,H2|T], ) :- H1 < H2, !, one_pass(, R).`

`one_pass([H1,H2|T], ) :- one_pass(, R).`

`one_pass(, ) :-`

`one_pass([], []).`

`?- one_pass([3,1,4,5,2], R).`

`R=[1,3,4,2,5];`

`no.`

alegeti intrebarea care adauga elementul "a" la sfarsitul unei liste diferenta ce continue elementele 1,2,3

`q(X,SL,[X|EL],SL,EL).`

`q(a,[1,2,3|L],L,RS,[]).`

Alegeți întrebarea care adaugă elementul "a" la sfârșitul unei liste diferență ce conține elementele 1, 2 și 3.

`q(X, SL, [X|EL], SL, EL).`

Select one or more:

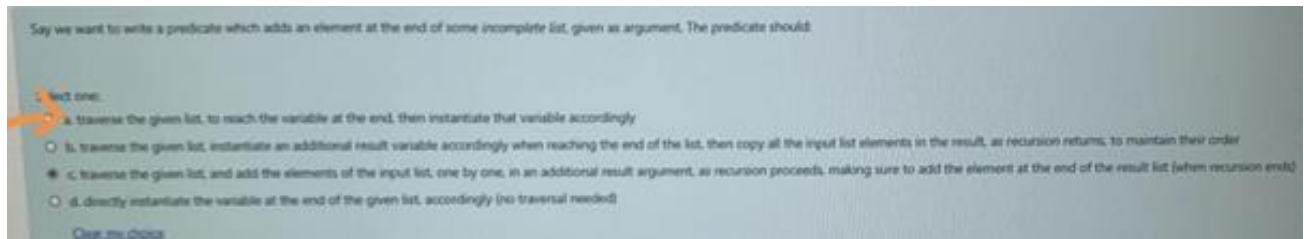
☐ a. `?-q(a, [1,2,3], [], RS,RE).`

☐ b. `?-q(a, [1,2,3|_], _, RS,RE).`

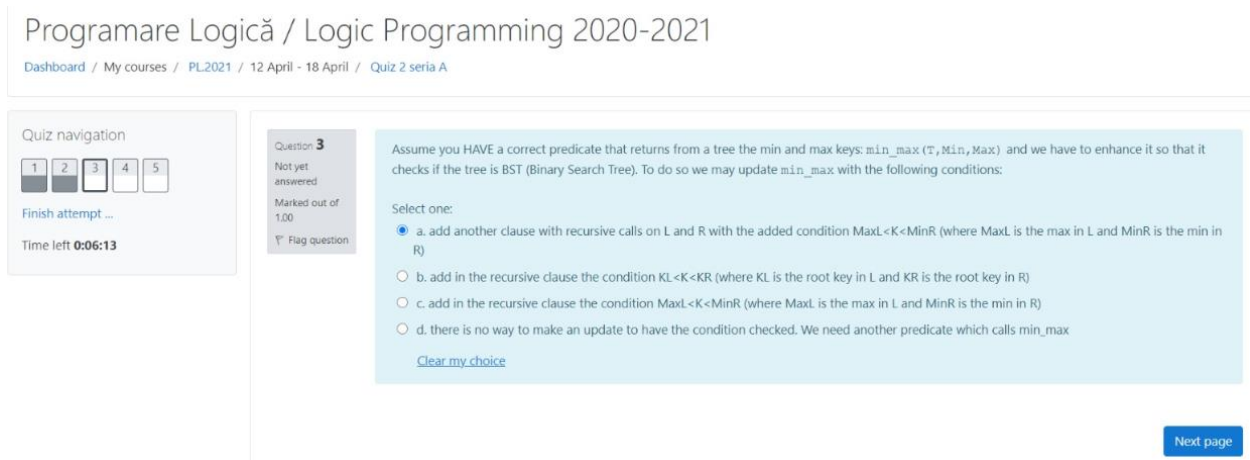
☐ c. `?-q(a, [1,2,3|_], [], RE).`

☒ d. `?-q(a, [1,2,3|L], L, RS,[]).` ✓

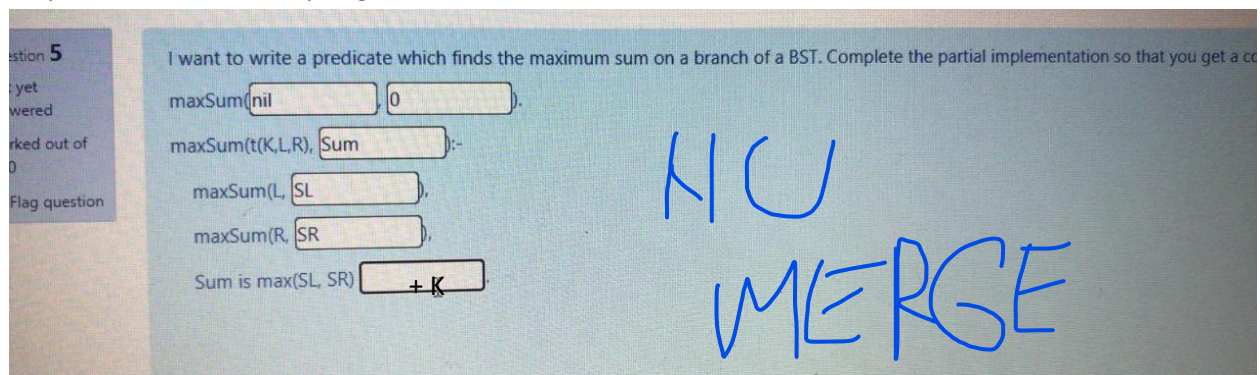
Say we want to write a predicate which adds an element at the end of some incomplete list given as an argument. The predicate should:



Assume you have a correct predicate that returns from a tree the min and max keys:  $\text{min\_max}(T, \text{Min}, \text{Max})$  and we have to enhance it so that it checks if the tree is BST. To do so we may update  $\text{min\_max}$  with the following conditions:



I want to write a predicate which finds the maximum sum on branch of a BST. Complete the partial implementation so that you get....





I want to write a predicate which finds the sum of all leaf nodes in a BST. Complete the partial implementation so that you get a correct predicate.

### Programare Logică / Logic Programming 2020-2021

Dashboard / My courses / PL2021 / 12 April - 18 April / Quiz 2 seria A

Quiz navigation

12345

Finish attempt ...

Time left 0:02:34

Question 5

Not yet answered

Marked out of 1.00

Flag question

I want to write a predicate which finds the sum of all leaf nodes in a BST. Complete the partial implementation so that you get a correct predicate:

```
sumLeaf(nil, 0).
sumLeaf(t(K,nil), nil, K).
sumLeaf(t(K,L,R), Sum):-
    sumLeaf(L, SL),
    sumLeaf(R, SR),
    Sum is SL + SR.
```

Finish attempt ...

The predicate below implements bubble sort(where append/3 has the regular, correct implementation). How many calls does the second append make to get the sorted array for the input [1,2,5,4,3]

Photos - WhatsApp Image 2021-06-05 at 13:41:28 (1).jpeg

See all photos + Add to

Edit & Create Share

ation, June 2020 / Final Examination - Part 1

3

ed  
out of  
an

The predicate below implements bubble sort (where append/3 has the regular, correct implementation). How many calls does the second append make to get the sorted array for the input [1,2,5,4,3].

```
bubble_sort(In, Out) :-
    append(X, [A, B|Y], In),
    A > B, !,
    append(X, [B, A|Y], Int),
    bubble_sort(Int, Out).
bubble_sort(In, In).
```

Select one:

☐ a. 4

☒ b. 3

☐ c. 2

☐ d. 5

[Clear my choice](#)

Next page

Type here to search

ENG 19:09 07.06.2021